When does it make sense to use Spring Cloud Kubernetes? As one example, Spring Cloud Gateway could be enhanced with Spring Cloud Kubernetes to get more control over service discovery and load balancing, including automatic registration of new routes based on Services metadata and the choice of load-balancing strategy. In this case, you could rely on the Spring Cloud Kubernetes Discovery Server component, limiting the need for Kubernetes API access to the discovery server.

Spring Cloud Kubernetes really shines when it comes to implementing Kubernetes controller applications to accomplish administrative tasks within the cluster. For example, you could implement a controller that monitors when ConfigMaps or Secrets change and then triggers a configuration refresh on the application using them. As a matter of fact, the Spring team used Spring Cloud Kubernetes to build a controller that does precisely that: the Configuration Watcher.

NOTE Spring Cloud Kubernetes Configuration Watcher is available as a container image on Docker Hub. If you'd like to know more about how it works and how to deploy it, you can refer to the official documentation (https://spring.io/projects/spring-cloud-kubernetes).

Besides the Configuration Watcher, Spring Cloud Kubernetes provides other convenient off-the-shelf applications for addressing common concerns of distributed systems in Kubernetes. One of them is a configuration server built on top of Spring Cloud Config and extending its functionality to support reading configuration data from ConfigMaps and Secrets. It's called Spring Cloud Kubernetes Config Server.

You can use this application directly (the container image is published on Docker Hub) and deploy it on Kubernetes following the instructions provided in the official documentation (https://spring.io/projects/spring-cloud-kubernetes).